

C++ - Module 09

STL

Özet: Bu doküman, C++ modüllerinin 09. Modülüne ait alıştırmaları içermektedir.

Versiyon: 3.1

İçindekiler

- I. Giriş — 2
 - II. Genel kurallar — 3
 - III. Modüle özgü kurallar — 6
 - IV. AI Talimatları — 7
 - V. Exercise 00: Bitcoin Exchange — 9
 - VI. Exercise 01: Reverse Polish Notation — 11
 - VII. Exercise 02: PmergeMe — 13
 - VIII. Teslim ve akran değerlendirmesi — 16
-

Chapter I — Giriş

C++, Bjarne Stroustrup tarafından C programlama dilinin bir uzantısı olarak yaratılmış, genel amaçlı bir programlama dilidir; ya da "Sınıflarla C" olarak da bilinir (kaynak: Wikipedia).

Bu modüllerin amacı sizi **Nesne Yönelimli Programlamaya** (Object-Oriented Programming) tanıtmaktır.

Bu, C++ yolculuğunuzun başlangıç noktası olacak. OOP öğrenmek için pek çok dil tavsiye edilir. Biz C++'ı seçtik, çünkü eski dostunuz C'den türemiştir.

Bu karmaşık bir dil olduğundan ve işleri basit tutmak adına, kodunuz C++98 standardına uygun olacaktır.

Modern C++'ın pek çok açıdan oldukça farklı olduğunun farkındayız. Dolayısıyla yetkin bir C++ geliştiricisi olmak istiyorsanız, 42 Common Core'dan sonra daha ileri gitmek size kalmış!

Chapter II — Genel kurallar

Derleme

- Kodunuzu `c++` ve `-Wall -Wextra -Werror` flag'leriyle derleyin.
- Kodunuz `-std=c++98` flag'i eklendiğinde de derlenebilmelidir.

Biçimlendirme ve adlandırma kuralları

- Alıştırma dizinleri şu şekilde adlandırılacak: `ex00`, `ex01`, ..., `exn`
- Dosyalarınızı, sınıflarınızı, fonksiyonlarınızı, üye fonksiyonlarınızı ve attribute'lerinizi yönergelerde istenildiği gibi adlandırın.
- Sınıf isimlerini **UpperCamelCase** formatında yazın. Sınıf kodunu içeren dosyalar her zaman sınıf ismine göre adlandırılacaktır. Örneğin: `ClassName.hpp/ClassName.h`, `ClassName.cpp`, ya da `ClassName.hpp`. Dolayısıyla, "BrickWall" (tuğla duvar) adında bir sınıfın tanımını içeren bir header dosyanız varsa, ismi `BrickWall.hpp` olacaktır.
- Aksi belirtilmedikçe, her çıktı mesajı bir newline karakteri ile bitmeli ve standart çıktıya (standard output) gösterilmelidir.
- Norminette'e elveda! C++ modüllerinde herhangi bir kodlama stili zorunlu kılınmamıştır. Kendi favori stilinizi takip edebilirsiniz. Ancak unutmayın ki ekran değerlendiricilerinizin anlayamadığı kod, not veremeyecekleri koddur. Temiz ve okunabilir kod yazmak için elinizden geleni yapın.

İzin verilenler / Yasaklananlar

Artık C ile kod yazmıyorsunuz. C++ zamanı! Bu nedenle:

- Standart kütüphaneden hemen hemen her şeyi kullanmanıza izin verilir. Dolayısıyla, alıştığınız şeylere bağlı kalmak yerine, alışkın olduğunuz C fonksiyonlarının C++'a özgü versiyonlarını mümkün olduğunca kullanmak akıllıca olacaktır.
- Ancak, başka herhangi bir harici kütüphane kullanamazsınız. Bu, C++11'in (ve türevlerinin) ve Boost kütüphanelerinin yasak olduğu anlamına gelir. Şu fonksiyonlar da yasaktır: `*printf()`, `*alloc()` ve `free()`. Bunları kullanırsanız notunuz 0 olur, hepsi bu kadar.
- Aksi açıkça belirtilmedikçe, `using namespace <ns_name>` ve `friend` anahtar kelimeleri yasaktır. Aksi takdirde notunuz -42 olur.
- STL'i yalnızca Module 08 ve 09'da kullanmanıza izin verilir. Bu, o zamana kadar Container'ların (vector/list/map vb.) ve Algorithm'lerin (`<algorithm>` header'ını içermeyi gerektiren her şey) yasak olduğu anlamına gelir. Aksi takdirde notunuz -42 olur.

Birkaç tasarım gereksinimi

- Bellek sızıntısı (memory leakage) C++'ta da yaşanır. `new` anahtar kelimesini kullanarak bellek ayırdığınızda, bellek sızıntılarından (memory leaks) kaçınılmalısınız.

- Module 02'den Module 09'a kadar, sınıflarınız aksi açıkça belirtilmedikçe **Orthodox Canonical Formda** tasarlanmalıdır.
- Bir header dosyasına konulan herhangi bir fonksiyon implementasyonu (function template'ler hariç) alıştırma için 0 anlamına gelir.
- Header'larınızın her birini diğerlerinden bağımsız olarak kullanabilmelisiniz. Dolayısıyla, ihtiyaç duydukları tüm bağımlılıkları include etmelidirler. Ancak, include guard'lar ekleyerek çift include (double inclusion) sorunundan kaçınmalısınız. Aksi takdirde notunuz 0 olur.

Beni oku (Read me)

- İhtiyaç duyarsanız bazı ek dosyalar ekleyebilirsiniz (örn. kodunuzu bölmek için). Bu ödevler bir program tarafından kontrol edilmediğinden, zorunlu dosyaları teslim ettiğiniz sürece bunu yapmaktan çekinmeyin.
- Bazen, bir alıştırmanın yönergeleri kısa görünür ama örnekler, talimatlarda açıkça yazılmamış gereksinimleri gösterebilir.
- Başlamadan önce her modülü baştan sona okuyun! Gerçekten yapın.
- Odin aşkına, Thor aşkına! Beyninizi kullanın!!!

Uyarı: C++ projeleri için Makefile konusunda, C'deki ile aynı kurallar geçerlidir (Norm'un Makefile bölümüne bakınız).

İpucu: Pek çok sınıf implement etmeniz gerekecek. Bu sıkıcı görünebilir, tabii favori metin editörünüzü script'leyebiliyorsanız durum değişir.

Bilgi: Alıştırmaları tamamlamak için size belirli bir özgürlük tanınıyor. Ancak, zorunlu kurallara uyun ve tembellik etmeyin. Aksi takdirde pek çok faydalı bilgiyi kaçırmış olursunuz! Teorik kavramlar hakkında okumaktan çekinmeyin.

Chapter III — Modüle özgü kurallar

Bu modüldeki her alıştırmaı gerçekleştirmek için standart container'ları kullanmak **zorunludur**.

Bir container kullanıldıktan sonra, modülün geri kalanında onu kullanamazsınız.

Bilgi: Alıştırmaları yapmadan önce konuyu bütünüyle okumanız tavsiye edilir.

Uyarı: Exercise 02 hariç her alıştırma için en az bir container kullanmanız gerekmektedir; Exercise 02 iki container kullanımı gerektirmektedir.

Her program için, kaynak dosyalarınızı `-Wall`, `-Wextra` ve `-Werror` flag'leriyle gerekli çıktıya derleyecek bir `Makefile` teslim etmelisiniz.

`c++` kullanmak zorundasınız ve Makefile'ınız relink yapmamalıdır.

Makefile'iniz en az şu kuralları içermelidir: \$(NAME), all, clean, fclean ve re.

Chapter IV — AI Talimatları

Bağlam (Context)

Bu proje, 42 eğitiminizin temel yapı taşlarını keşfetmenize yardımcı olmak için tasarlanmıştır.

Temel bilgi ve becerileri doğru şekilde yerleştirmek için, AI araçlarını ve desteğini kullanma konusunda düşünceli bir yaklaşım benimsemek esastır.

Gerçek temel öğrenme, zorluk, tekrar ve akranlarla öğrenme alışverişi yoluyla gerçek bir entelektüel çaba gerektirir.

AI hakkındaki tutumumuza dair daha kapsamlı bir genel bakış için — bir öğrenme aracı olarak, 42 eğitiminin bir parçası olarak ve iş piyasasında bir beklenti olarak — lütfen intranetteki ilgili SSS'ye (FAQ) bakınız.

Ana mesaj

- Kestirmelere başvurmadan sağlam temeller inşa edin.
- Tech & power skills'inizi gerçekten geliştirin.
- Gerçek akran öğrenmeyi deneyimleyin, nasıl öğreneceğinizi ve yeni problemleri nasıl çözeceğinizi öğrenmeye başlayın.
- Öğrenme yolculuğu sonuçtan daha önemlidir.
- AI ile ilişkili riskler hakkında bilgi edinin ve yaygın tuzaklardan kaçınmak için etkili kontrol pratikleri ve karşı önlemler geliştirin.

Öğrenci kuralları

- Görevlerinize, özellikle AI'ya başvurmadan önce, akıl yürütme uygulamalısınız.
- AI'dan doğrudan cevap istememelisiniz.
- 42'nin AI konusundaki genel yaklaşımını öğrenmelisiniz.

Aşama çıktıları

Bu temel aşama içinde şu çıktıları elde edeceksiniz:

- Düzgün tech ve kodlama temelleri kazanmak.
- Bu aşamada AI'nin neden ve nasıl tehlikeli olabileceğini bilmek.

Yorumlar ve örnek

- Evet, AI'nin var olduğunu biliyoruz — ve evet, projelerinizi çözebilir. Ama siz burada öğrenmek için varsınız, AI'nin öğrendiğini kanıtlamak için değil. AI'nin verilen

problemi çözebildiğini göstermek için sadece zamanınızı (veya bizimkini) boşa harcamayın.

- 42'de öğrenmek, cevabı bilmekle ilgili değildir — bir cevap bulma yeteneği geliştirmekle ilgilidir. AI size cevabı doğrudan verir, ama bu sizin kendi akıl yürütmenizi inşa etmenizi engeller. Akıl yürütme zaman, çaba gerektirir ve başarısızlığı da içerir. Başarıya giden yol kolay olması gerektiği şekilde tasarlanmamıştır.
- Sınavlar sırasında AI'nin mevcut olmadığını unutmayın — internet yok, akıllı telefon yok, vb. Öğrenme sürecinizde AI'ye fazlasıyla güvenip güvenmediğinizi hızlıca fark edeceksiniz.
- Akran öğrenmesi sizi farklı fikirlere ve yaklaşımlara maruz bırakır, kişilerarası becerilerinizi ve farklı şekillerde düşünme yeteneğinizi geliştirir. Bu, sadece bir botla sohbet etmekten çok daha değerlidir. O yüzden çekinmeyin — konuşun, sorular sorun ve birlikte öğrenin!
- Evet, AI müfredatın bir parçası olacak — hem bir öğrenme aracı hem de kendi başına bir konu olarak. Hatta kendi AI yazılımınızı inşa etme şansınız bile olacak. Gececeğiniz crescendo yaklaşımımız hakkında daha fazla bilgi edinmek için intranette mevcut dokümantasyona bakın.

✓ **İyi pratik:** Yeni bir kavramda takılı kaldım. Yakınımdaki birine bu konuya nasıl yaklaştığını soruyorum. 10 dakika konuşuyoruz — ve birden anlıyorum. Kavrادم.

✗ **Kötü pratik:** Gizlice AI kullanıyorum, doğru görünen bir kod kopyalıyorum. Akran değerlendirmesi sırasında hiçbir şeyi açıklayamıyorum. Başarısız oluyorum. Sınavda — AI yok — yine takılıyorum. Başarısız oluyorum.

Chapter V — Exercise 00: Bitcoin Exchange

Exercise	00
	Bitcoin Exchange
Directory	<code>ex00/</code>
Teslim edilecek dosyalar	<code>Makefile</code> , <code>main.cpp</code> , <code>BitcoinExchange.{cpp, hpp}</code>
Yasaklı	Yok

Belirli bir tarihte belirli bir miktarda bitcoin'in değerini çıktı olarak veren bir program oluşturmanız gerekmektedir.

Bu program, zaman içindeki bitcoin fiyatını temsil eden csv formatında bir veritabanı kullanmalıdır. Bu veritabanı bu konu ile birlikte sağlanmaktadır.

Program, değeriendirilecek farklı fiyatları/tarihleri saklayan ikinci bir veritabanını girdi olarak alacaktır.

Programınız şu kurallara uymalıdır:

- Program adı `btc`'dir.
- Programınız argüman olarak bir dosya almalıdır.
- Bu dosyadaki her satır şu formatı kullanmalıdır: `"tarih | değeri"`.
- Geçerli bir tarih her zaman şu formatta olacaktır: Yıl-Ay-Gün.
- Geçerli bir değeri, 0 ile 1000 arasında bir float ya da pozitif bir integer olmalıdır.

Uyarı: Bu alıştırmayı onaylamak için kodunuzda en az bir container kullanmanız gerekmektedir. Olası hataları uygun bir hata mesajı ile ele almalısınız.

input.txt dosyası örneği:

```
$> head input.txt
date | value
2011-01-03 | 3
2011-01-03 | 2
2011-01-03 | 1
2011-01-03 | 1.2
2011-01-09 | 1
2012-01-11 | -1
2001-42-42
2012-01-11 | 1
2012-01-11 | 2147483648
$>
```

Programınız, input dosyanızdaki değeri kullanacaktır.

Programınız, standart çıktıya, değeri veritabanınızda belirtilen tarihe göre döviz kuru ile çarpılmış sonucunu göstermelidir.

İpucu: Girdide kullanılan tarih veritabanınızda mevcut değerse, veritabanınızdaki en yakın tarihi kullanmalısınız. Üst tarihi değeri, alt tarihi kullandığınıza dikkat edin.

Programın kullanım örneği:

```
$> ./btc
Error: could not open file.
$> ./btc input.txt
2011-01-03 => 3 = 0.9
2011-01-03 => 2 = 0.6
2011-01-03 => 1 = 0.3
2011-01-03 => 1.2 = 0.36
2011-01-09 => 1 = 0.32
```

```
Error: not a positive number.  
Error: bad input => 2001-42-42  
2012-01-11 => 1 = 7.1  
Error: too large a number.  
$>
```

Uyarı: Bu alıştırımayı onaylamak için kullandığınız container(lar) bu modülün geri kalanında artık kullanılamaz.

Chapter VI — Exercise 01: Reverse Polish Notation

Exercise	01
	RPN
Directory	ex01/
Teslim edilecek dosyalar	Makefile, main.cpp, RPN.{cpp, hpp}
Yasaklı	Yok

Bu kısıtlamalarla bir program oluşturmanız gerekmektedir:

- Program adı `RPN`'dir.
- Programınız, argüman olarak ters Lehçe (inverted Polish) matematiksel bir ifade almalıdır.
- Bu işlemde kullanılan ve argüman olarak geçilen sayılar her zaman 10'dan küçük olacaktır. Hesaplamanın kendisi ve sonucu bu kurala tabi değildir.
- Programınız bu ifadeyi işlemeli ve doğru sonucu standart çıktıya yazdırmalıdır.
- Program çalışması sırasında bir hata oluşursa, standart hataya (standard error) bir hata mesajı gösterilmelidir.
- Programınız şu token'larla işlem yapabilmelidir: `" + - / * "`.

Uyarı: Bu alıştırımayı onaylamak için kodunuzda en az bir container kullanmanız gerekmektedir.

Bilgi: Parantezleri veya ondalıklı sayıları ele almanız gerekmiyor.

Standart kullanım örneği:

```
$> ./RPN "8 9 * 9 - 9 - 9 - 4 - 1 +"  
42  
$> ./RPN "7 7 * 7 -"
```

```
42
$> ./RPN "1 2 * 2 / 2 * 2 4 - +"
0
$> ./RPN "(1 + 1)"
Error
$>
```

Uyarı: Bir önceki alıştırmada kullandığınız container(lar) burada yasaktır. Bu alıştırmayı onaylamak için kullandığınız container(lar) modülün geri kalanında kullanılamaz.

Chapter VII — Exercise 02: PmergeMe

Exercise	02
	PmergeMe
Directory	ex02/
Teslim edilecek dosyalar	Makefile, main.cpp, PmergeMe.{cpp, hpp}
Yasaklı	Yok

Bu kısıtlamalarla bir program oluşturmanız gerekmektedir:

- Program adı `PmergeMe`'dir.
- Programınız, argüman olarak pozitif bir integer dizisi kullanabilmelidir.
- Programınız, pozitif integer dizisini sıralamak için merge-insert sort algoritmasını kullanmalıdır.

Uyarı: Açıklamak gerekirse, evet, Ford-Johnson algoritmasını kullanmanız gerekiyor. (kaynak: Art Of Computer Programming, Cilt 3. Merge Insertion, Sayfa 184.)

- Program çalışması sırasında bir hata oluşursa, standart hataya (standard error) bir hata mesajı gösterilmelidir.

Uyarı: Bu alıştırmayı onaylamak için kodunuzda en az iki farklı container kullanmanız gerekmektedir. Programınız en az 3000 farklı integer'ı işleyebilmelidir.

İpucu: Her container için algoritmanızı ayrı ayrı implement etmeniz ve dolayısıyla genel bir fonksiyon kullanmaktan kaçınmanız şiddetle tavsiye edilir.

Standart çıktıda satır satır göstermeniz gereken bilgilere ilişkin ek yönergeler:

- Birinci satırda sıralanmamış pozitif integer dizisinin ardından açıklayıcı bir metin

göstermelisiniz.

- İkinci satırda sıralanmış pozitif integer dizisinin ardından açıklayıcı bir metin göstermelisiniz.
- Üçüncü satırda, pozitif integer dizisini sıralamak için kullanılan birinci container'ı belirterek algoritmanızın harcadığı süreyi gösteren açıklayıcı bir mesaj göstermelisiniz.
- Son satırda, pozitif integer dizisini sıralamak için kullanılan ikinci container'ı belirterek algoritmanızın harcadığı süreyi gösteren açıklayıcı bir metin göstermelisiniz.

Bilgi: Sıralama işleminizi gerçekleştirmek için harcanan sürenin gösterim formatı serbesttir, ancak seçilen hassasiyet, kullanılan iki container arasındaki farkı açıkça gösterebilmelidir.

Standart kullanım örneği:

```
$> ./PmergeMe 3 5 9 7 4
Before: 3 5 9 7 4
After:  3 4 5 7 9
Time to process a range of 5 elements with std::[...] : 0.00031 us
Time to process a range of 5 elements with std::[...] : 0.00014 us
$> ./PmergeMe `shuf -i 1-100000 -n 3000 | tr "\n" " "`
Before: 141 79 526 321 [...]
After:  79 141 321 526 [...]
Time to process a range of 3000 elements with std::[...] : 62.14389 us
Time to process a range of 3000 elements with std::[...] : 69.27212 us
$> ./PmergeMe "-1" "2"
Error
$> # OSX KULLANICILARI İÇİN:
$> ./PmergeMe `jot -r 3000 1 100000 | tr '\n' ' '`
[...]
$>
```

İpucu: Bu örnekteki süre gösterimi kasıtlı olarak tuhaftır. Elbette, hem sıralama hem de veri yönetimi dahil tüm işlemlerinizi için harcanan süreyi göstermeniz gerekmektedir.

Uyarı: Önceki alıştırmalarda kullandığınız container(lar) burada yasaktır.

Bilgi: Yinelenen (duplicate) değerlerle ilgili hataların yönetimi sizin takdirinize bırakılmıştır.

Chapter VIII — Teslim ve akran değerlendirmesi

Ödevinizi her zamanki gibi Git repository'nize teslim edin. Savunma (defense) sırasında yalnızca repository'niz içindeki çalışma değerlendirilecektir. Klasör ve dosya isimlerinizin

doğru olduğundan emin olmak için kontrol etmekten çekinmeyin.

Değerlendirme sırasında, projede kısa bir **modifikasyon** ara sıra talep edilebilir. Bu, küçük bir davranış değişikliği, yazılacak veya yeniden yazılacak birkaç satır kod, ya da eklemesi kolay bir özellik içerebilir.

Bu adım her proje için geçerli olmayabilir, ancak değerlendirme yönergelerinde belirtilmişse buna hazırlıklı olmalısınız.

Bu adım, projenin belirli bir bölümünü gerçekten anlayıp anlamadığınızı doğrulamak içindir. Modifikasyon, seçtiğiniz herhangi bir geliştirme ortamında (örn. kendi kullandığınız kurulum) yapılabilir ve değerlendirmenin bir parçası olarak belirli bir süre tanımlanmadıkça birkaç dakika içinde gerçekleştirilebilir olmalıdır.

Örneğin, bir fonksiyonda veya script'te küçük bir güncelleme yapmanız, bir görüntüyü (display) değiştirmeniz, ya da yeni bilgi saklamak için bir veri yapısını ayarlamanız vb. istenebilir.

Detaylar (kapsam, hedef vb.) değerlendirme yönergelerinde belirtilecektir ve aynı proje için bir değerlendirmeden diğerine farklılık gösterebilir.